NODE JS

01#########

What is Node js

1. Node js is runtime environment for javaScript.
2. Node we can run JavaScript outside to the browser

Scripting Languages

Like server, windows, mac , linux etc

1. JavaScript run inside the browser.

Where we need JS outside browser

1. Web browser Run on client computer.
2. So Javascript do not handle things on server.
3. So we can not connect javascript with Database directly
4. Can not handle File system etc
5. So we have to run javascript on browser and Node help for this

Why need Node

1. Unified Language
2. High Performance
3. Good for Real-time Applications
4. Good for Data Streaming
5. Etc

How to use it full stack and FE technology

1. We make API in node js.
2. Integrate APIs with FE technology.
3. BE+FE = Full Stack.

Java (Programming Language)

History Of Node js

1. First Release :- May 27, 2009
2. Current Version: V24.21.0
3. Written in JS, C and c++

02#########

Node.js Setup on MacOS

Step 1: Download Node.js

1. Go to the official Node.js website:
2. 👉 https://nodejs.org/en
3. Click on **Download Node.js (LTS)** version.
4. *(LTS means Long Term Support – best for most users.)*

Scripting Languages

Step 2: Install Node.js

1. Open the downloaded file and complete the installation steps.

2. Once installed, Node.js and **npm (Node Package Manager)** will be available.

Check Installed Versions

Open **Terminal**.

Type the following commands to verify installation:

node -v

Mac OS

npm -v

This will show you the version numbers of Node.js and npm installed.

Node.js Setup on Windows

Step 1: Download Node.js

1. Go to:
2. 👉 https://nodejs.org/en
3. Click on the **Download Node.js (LTS)** version.

Step 2: Install Node.js

1. Run the downloaded setup file and complete the installation.
2. This installs both **Node.js** and **npm**.

Step 3: Check Installed Versions

1. Open **Command Prompt (CMD)**.
2. Run these commands:

Java (Programming Language)

node -v

npm -v

You should see version numbers if everything was installed correctly.

Install VS Code Editor

To write and run Node.js code smoothly, install **Visual Studio Code** (VS Code) — a lightweight code editor.

For macOS:

1. Visit: https://code.visualstudio.com/
2. Click **Download for macOS** and follow the install instructions.

Scripting Languages

For Windows:

1. Same site: https://code.visualstudio.com/
2. Click **Download for Windows**.

Once installed, open VS Code and start coding in JavaScript and Node.js easily.

3#########

Create Folder and File

Create Folder

On Desktop, create a folder manually (e.g., NodeApp)

or

Open Terminal and run:

mkdir NodeApp

cd NodeApp

Open Folder in VS Code

Using VS Code directly

Open **VS Code**

Go to **File > Open Folder**

Select your NodeApp folder

Using Terminal

code .

This opens the current folder in VS Code (if code command is configured).

Create a JavaScript File

1. Create a file named app.js

Node.js runs JavaScript files, so .js extension is required.

Scripting Languages

Write Some Basic Code

Basic Output

```
console.log(20 + 20); // Output: 40
```

Function Example

```
function fruit(item) {
```

```
console.log("Fruit name is " + item);
```

Java (Programming Language)

```
}
fruit("apple");
```

Variable Example

```
var a = 10;
var b = 20;

console.log(a + b); // Output: 30
```

Run Node.js Code

Steps:

1. Open terminal
2. Navigate to your project folder (if not already)
3. Run:

```
node app.js
```

File System Example (Just to Show)

```
var fs = require("fs"); // Import file system module
```

Operating Systems

```
fs.writeFileSync("anil.txt", "My name is Anil");
```

This creates a file anil.txt and writes the content inside it.

REPL (Read Eval Print Loop)

1. Just type node in terminal to enter REPL mode
2. Then type JS code line by line

> 2 + 2

4

> .exit

4#########

What Are Modules and Their Types?

Core Modules

1. Core modules are already available in Node.js.
2. You do not need to install them.

```
const fs = require("fs");
```

Learn Scripting

```
const os = require("os"); // import os module


fs.writeFileSync("dummy.txt", "trying with modules");
// Create a file with name and write content in it


console.log(os.platform()); // Print the platform of the OS
console.log(os.hostname()); // Print the hostname of the OS
console.log(os.cpus()); // Print the CPU information of the OS
```

Third-party Modules

You can install them from third party.

Custom Modules

You create them by yourself.

Earn a CS Degree

What Are Core Modules?

1. Core modules are modules that are already included in Node.js.
2. No need to install them separately. Just use require() to import them.

What Are Global Objects?

1. Global object is an object that is available in all modules by default.
2. You do not need to import or require it to use it.

You can import console forcefully:

Learn Java

```
const { log } = require("console"); // import console module


log("custom log");
```

Difference Between console.log in JS and Node.js

The difference is:

1. In **JavaScript** → console.log prints on the browser console.
2. In **Node.js** → console.log prints on the terminal.

5########

Why Do We Need a Server?

JavaScript by default runs in the browser.

But when we want to run JavaScript on the server, we need Node.js.

Now, if you write some code in Node.js — how can you access that code in the browser?

It's important to understand this:

If you want to see your Node.js output in the browser, you need to set up a server.

Write Code to Create a Server

Node.js provides a built-in package called http to create servers.

You don't need to install it separately.

Here's the basic code to create a server:

```
const http = require("http");

http.createServer().listen(4800);
```

How to Run the Code

Open your terminal and run the file using:

node basics.js

How to Check the Output in the Browser

Open your browser and type:

http://localhost:4800

You can also check the response in:

Inspect → Network tab

Understanding Request and Response Parameters

If you want your server to return some data to the browser, this is how it works:

1. The **client** (browser) sends a request.
2. The **server** responds to that request.

In Node.js, you can handle this using the http.createServer() method, which accepts a callback function with request and response parameters.

```
http
.createServer((req, resp) => {
resp.write("<h1>Hello, this is Anil Sidhu</h1>");
resp.end("Hello"); // Ends the response
})
.listen(4800);
```

Now, when you visit http://localhost:4800, you will see the message in the browser.

6#########

What is an External Package?

An external package is a package we install from a third party (someone else wrote it).

We use it to add **extra functionality** to our project without writing everything ourselves.

Example:

nodemon — a package that automatically restarts the server when code changes.

Without this, we would have to stop and start the server manually after every change.

How to Install an External Package

Example: Install the colors package from https://www.npmjs.com/package/colors

Open the terminal and run:

npm install colors

Try Cloud Tools

or

npm i colors

Example of Using an External Package

```
const colors = require("colors");


console.log(colors.red("Hi, this is Anil Sidhu"));
console.log(colors.green("Hi, this is Anil Sidhu"));
console.log(colors.yellow("Hi, this is Anil Sidhu"));
```

To run this code, type in the terminal:

node colors.js

Follow Stars

What is package.json?

package.json is a file that contains details about your project, such as:

1. Project name
2. Project version
3. External packages used (dependencies)
4. Project description
5. Author
6. License

What is package-lock.json?

package-lock.json is a file that contains detailed information about the exact versions of every installed package **and its dependencies** (internal packages).

Learn Scripting

It ensures that everyone working on the project installs **the same package versions**.

What is node_modules?

node_modules is a folder where all installed external packages are stored.

How to Create a package.json File

Run this command in the terminal:

npm init

Earn a CS Degree

You will be asked to fill in some details:

package name: node-2025

version: (1.0.0)

description: node js tutorial in hindi 2025

entry point: (index.js)

git repository:

keywords:

author: Anil Sidhu

license: (ISC)

Important Notes

1. External packages are stored inside the node_modules folder.
2. **You should not push node_modules to GitHub** — because it is a very large folder.
3. Other developers can easily install the required packages by running:

Try Cloud Tools

npm install

This will read package.json and package-lock.json and install everything needed.

7#########

Why do we need the nodemon package?

When you're working with a Node.js server, every time you make a change in your server file, you have to:

1. **Manually stop** the server.
2. **Restart** it to see the changes.

This process becomes annoying during development

Solution: nodemon

1. nodemon automatically **watches your files**.
2. It **restarts the server** automatically whenever you make changes.

3. Saves **time** and improves **developer experience**.

How to Install Nodemon

Open your terminal and run:

npm install nodemon

How to Use Nodemon

nodemon app.js

Now the server will restart automatically every time you make changes to app.js

Can we create more than one server?

Yes, we can run multiple servers in Node.js — just make sure they run on **different ports**.

10#########

What is a Server Response?

1. When a **client** (like a browser) sends a request to a server (e.g., by visiting google.com), the **server processes the request** and sends back a **response**.
2. This response can be **HTML**, **JSON**, **plain text**, files, or anything else.

Java (Programming Language)

Example:

When you visit **google.com**, the server sends back HTML that builds the **Google homepage** on your screen.

Basic Example of Server Response

const http = require("http");

```
http
  .createServer((req, resp) => {
    resp.setHeader("Content-Type", "text/html"); // Tell browser the response is HTML
    resp.write("<h1>Hello Anil Sidhu</h1>");
    resp.end(); // End the response
  })
  .listen("4800");
```

How to run:

1. Save as app.js
2. Run: nodemon app.js or node app.js
3. Open your browser and go to: http://localhost:4800

Passing Variables and Functions in Response

You can also include **variables**, **functions**, or **dynamic values** (like current time):

```
const http = require("http");


const age = 29;
Web Design & Development


http
  .createServer((req, resp) => {
    resp.setHeader("Content-Type", "text/html");


    resp.write(`
```

```
    <html>
      <head>
        <title>Code Step by Step</title>
      </head>
      <body>
        <h1>Hello Anil Sidhu</h1>
        <h2>Age: ${age}</h2>
        <h2>Current Date & Time: ${Date()}</h2>
      </body>
    </html>
  `);

  resp.end();
})
.listen(4800);
```

1. resp.setHeader("Content-Type", "text/html"): Tells the browser that we're sending back HTML.
2. Template literals (` `): Used to inject JavaScript variables like age and Date() into the HTML.

11#########

Create Server in simpleAPI.js

We'll use Node.js http module to create a basic server.

Learn to Code

Define Static API Data

```
const userData = [
{
name: "Anil",
age: 30,
email: "anil@test.com",
},
{
name: "sam",
age: 20,
email: "sam@test.com",
},
{
name: "peter",
```

Learn Java

```
age: 30,
email: "peter@test.com",
},
];
```

This is sample static data to simulate API response.

Use This Data in Server Response

```
const http = require("http");
```

```
http
.createServer((req, resp) => {
resp.setHeader("Content-Type", "application/json"); // Set the content type to JSON
Learn Scripting
resp.write(JSON.stringify(userData)); // Convert the userData array to a JSON string
resp.end(); // End the response
})
.listen(6000);
```

How to Test the API

Run the file using:

nodemon simpleAPI.js

Open browser and go to:

Browse Art Prints

http://localhost:6000

Test API using Thunder Client

You can also use Thunder Client (VS Code extension) or Postman:

1. Method: GET
2. URL: http://localhost:6000

You will get the JSON response.

13#########

What is Command Line Input?

Sometimes, we want to pass values **directly while running a file** from the terminal — like a username or a dynamic port number.

Create File: command-line-input.js

nodemon command-line-input.js anil sidhu

This runs the file and passes "anil" and "sidhu" as input arguments.

How to Get Command Line Input

```
const arg = process.argv;


console.log("-------", arg);
```

Output:------- [

'C:\\Program Files\\nodejs\\node.exe',

'C:\\Users\\sheet\\OneDrive\\Desktop\\turorial\\node 2025\\command-line-input.js',

'anil',

'sidhu'

]

 process.argv is an array:

1. Index 0 → Node path
2. Index 1 → File path
3. Index 2 onwards → Actual input values

Learn Java

Access Individual Input

```
console.log("-------", arg[3]);
```

Output:

------- sidhu

Note: Space-separated values are stored as separate elements in the array.

Example Use Case: Dynamic Port

```
const http = require("http");
const arg = process.argv;
const port = arg[2];

http
  .createServer((req, resp) => {
    resp.write("test input from cmd");
    resp.end();
  })
  .listen(port);
```

Run this from Terminal:

nodemon command-line-input.js 4100

This will start a server

14#########

Why load an HTML file with fs?

1. Instead of writing HTML as a string in resp.write(), it's more scalable and readable to keep HTML in a separate .html file.
2. This approach is cleaner, especially for large or structured HTML.

Computer Drives & Storage

Server File (web.js)

```
const http = require("http");


http.createServer((req, resp) => {}).listen(3200);
```

1. Creates a valid HTTP server.
2. Listens on port 3200.

HTML File (web.html)

```
<!DOCTYPE html>
```

Operating Systems

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Web page</title>
  </head>
```

```
 <body>
  <h1>Web Page Heading</h1>
  <input type="text" />
  <button>Click</button>
 </body>
</html>
```

1. Well-formed HTML structure.
2. Proper title, meta tags, and UI elements.

Read and Load HTML File Using fs Module

```
const http = require("http");
```

Web Design & Development

```
const fs = require("fs");


http
 .createServer((req, resp) => {
  fs.readFile("html/web.html", "utf-8", (err, data) => {
   if (err) {
    resp.writeHead(500, {
     "Content-Type": "text/plain",
    });
    resp.write("internal server error");
    resp.end();
    return;
   }
```

```
    resp.writeHead(200, {
      "Content-Type": "text/html",
    });
    resp.write(data);
    resp.end();
```

Computer Drives & Storage

```
  });
}).listen(3200);
```

Correctly reads the file html/web.html (assuming the folder structure is correct).

1. Sets appropriate status codes (500 for error, 200 for success).
2. Handles errors gracefully.
3. Sends the HTML content as response.

Check Output

1. When you run node web.js or nodemon web.js, and visit http://localhost:3200, the browser will correctly show the HTML content from web.html.

15#########

Form inside JS file

You correctly created and served a basic HTML form directly via resp.write():

```
const http = require("http");
```

Learn Java

```
http
  .createServer((req, resp) => {
    resp.writeHead(200, {
      "Content-Type": "text/html",
    });
    resp.write(`
      <form action="/submit" method="POST">
        <input type="text"  placeholder="Enter username" name="name" />
        <input type="text"  placeholder="Enter email" name="email" />
        <button>Submit</button>
      </form>
    `);
    resp.end();
  })
  .listen(3200);
```

This works fine, though you're not handling the actual POST data yet — just confirming routing works.

Learn Scripting

Conditional Handling of /submit

```
if (req.url === "/") {
  // form page
} else if (req.url === "/submit") {
  // confirmation message
```

}

This conditional routing is correct for handling basic URL-based navigation.

Using a separate form.html file

<!DOCTYPE html>

Build a Website

<html lang="en">

<head>

<meta charset="UTF-8" />

<meta name="viewport" content="width=device-width, initial-scale=1.0" />

<title>Form page</title>

</head>

<body>

<form action="/submit" method="POST">

<input type="text" placeholder="Enter username" name="name" />

<input type="text" placeholder="Enter email" name="email" />

<button>Submit</button>

</form>

</body>

</html

Perfectly structured form with POST method and inputs.

Load HTML File via fs

const http = require('http');

Store Big Data

const fs = require('fs');

```
http.createServer((req, resp) => {
  fs.readFile('html/form.html', 'utf-8', (error, data) => {
    if (error) {
      resp.writeHead(500, { "content-type": 'text/plain' });
      resp.end('internal server error');
      return;
    }

    resp.writeHead(200, { "content-type": 'text/html' });

    if (req.url == '/') {
      resp.write(data);
    } else if (req.url == '/submit') {
      resp.write('<h1>Data submitted</h1>');
    }

    resp.end();
  });
}).listen(3200);
```

Everything is correct:

1. Loads HTML file when GET /
2. Shows confirmation message when /submit is requested
3. Responds with correct headers
4. Properly handles fs.readFile errors

16########

When a form is submitted using the POST method, data is sent in chunks. To handle this in Node.js, we use two important events: `on('data')` and `on('end')`.

hen a form is submitted using the POST method, data is sent in chunks. To handle this in Node.js, we use two important events: on('data') and on('end').

```
const http = require("http");

const fs = require("fs");

const querystring = require("querystring");


http

  .createServer((req, res) => {

    fs.readFile("html/web.html", "utf-8", (err, data) => {

      if (err) {

        res.writeHead(500, { "content-type": "text/plain" });

        res.end("internel srevre error");

        return;

      }

      res.writeHead(200, { "content-type": "text/html" });


      if (req.url == "/") {

        res.write(data);

      } else if (req.url == "/submit") {

        res.write(`<h1>the form has been submitted </h1>`);

        let formData = [];
```

```
      req.on("data", (chunk) => {
        formData.push(chunk);
      });
      req.on("end", () => {
        let fullFormData = Buffer.concat(formData).toString();
        let formatData = querystring.parse(fullFormData);
        console.log(formatData);
      });
    }
    res.end();
  });
})
.listen(7654);
```

Get data from request

We collect incoming data chunks into a buffer:

```
let dataBody = [];
req.on("data", (chunk) => {
  dataBody.push(chunk);
});
```

Handle data with Buffer

Once all data is received, we combine and convert it to a string:

```
req.on("end", () => {
  let rawData = Buffer.concat(dataBody).toString();
  console.log(rawData);
```

});

resp.write(`<h1> Data Submitted </h1>`);

Learn Scripting

Make request data readable

To parse this string into key-value pairs, use Node's built-in querystring module:

```
const querystring = require("querystring");


req.on("end", () => {
  let rawData = Buffer.concat(dataBody).toString();
  let readableData = querystring.parse(rawData);
  console.log(readableData);
});
```

Notes

1. on('data') collects the chunks
2. on('end') finalizes the data
3. querystring.parse() makes the data easier to use

17#########

Synchronous File Creation

```
let dataString =
  "My name is " +
  readableData.name +
```

```
  " and my email is " +
  readableData.email;

fs.writeFileSync("text/" + readableData.name + ".txt", dataString);
console.log("File Created");
```

1. dataString: combines name and email from the form.
2. writeFileSync: writes data to file in a blocking way (waits till complete).
3. "text/" + readableData.name + ".txt": file name is based on user input.

Asynchronous File Creation

```
fs.writeFile(
  "text/" + readableData.name + ".txt",
  dataString,
  "utf-8",
  (err) => {
    if (err) {
      resp.end("Internal Server Error");
      return false;
    } else {
      console.log("File Created");
    }
  }
);
```

1. writeFile: non-blocking (code runs while file is being created).
2. "utf-8": encoding type.
3. Callback handles success or error.

18#########

Main Function Execution

When Node.js runs code, it first registers the **main()** function in the **call stack**. Then other functions are called in order.

```
const x = 1;

const y = x + 2;

console.log("sum is " + y);
```

1. main() is added to the call stack automatically.
2. After console.log(), it exits the stack.
3. Only main() remains and exits last.

Example with a Function

```
const listLocations = (locations) => {
  locations.forEach((location) => {
Earn a CS Degree
    console.log(location);
  });
};


const myLocations = ["philly", "nyc"];


listLocations(myLocations);
```

Call Stack Flow:

1. listLocations enters call stack

2. forEach runs each item
3. console.log("philly")
4. Stack clears, then next item:
5. console.log("nyc")
6. Stack becomes empty

Learn to Code

Node APIs & setTimeout

Node APIs handle asynchronous code like setTimeout.

```
console.log("Starting up");

setTimeout(() => {
  console.log("Two Seconds");
}, 2000);

setTimeout(() => {
  console.log("Zero Seconds");
}, 0);

console.log("Finishing up");
```

Learn Scripting

Call Stack Output Order:

1. Starting up
2. Finishing up
3. (then Node APIs hold timers)

Event Loop then executes callbacks when the call stack is empty:

3. Zero Seconds

4. Two Seconds

Event Loop

1. Keeps checking if the **call stack** is empty.
2. If empty, moves code from **callback queue** to **call stack**.

19#########

Node.js is great for creating modular server-side applications. In this post, we'll create a simple project using **modules**, **routing**, and **form handling**.

Project Structure

Create 3 files:

1. root.js (main server file)
2. userForm.js (HTML form)
3. userDataSubmit.js (handle form data)

root.js (Main Server File)

```
const http = require("http");

const userForm = require("./userForm");

const userDataSubmit = require("./userDataSubmit");
```

http

```
  .createServer((req, resp) => {
    resp.writeHead(200, { "Content-Type": "text/html" });

    if (req.url === "/") {
      userForm(req, resp);
    } else if (req.url === "/submit") {
      userDataSubmit(req, resp);
    }

    resp.end();
  })
  .listen(3200);
```

This file:

1. Creates the HTTP server
2. Routes / to form and /submit to form handler
3. Sends HTML response based on the route

userForm.js (HTML Form)

```
function userForm(req, resp) {
  resp.write(`
    <form action="/submit" method="POST">
      <input type="text" placeholder="Enter username" name="name" />
      <input type="text" placeholder="Enter email" name="email" />
      <button>Submit</button>
    </form>
```

```
  `);
}

module.exports = userForm;
```

This file:

1. Displays a form with two input fields
2. Sends data to /submit via POST method

userDataSubmit.js (Handle Form Data)

```
const querystring = require("querystring");

function userDataSubmit(req, resp) {
  let dataBody = [];

  req.on("data", (chunk) => {
    dataBody.push(chunk);
  });

  req.on("end", () => {
    let rawData = Buffer.concat(dataBody).toString();
    let readableData = querystring.parse(rawData);

    let dataString =
      "My name is " +
      readableData.name +
      " and my email is " +
```

```
    readableData.email;


  console.log(dataString);
 });


 resp.write(`<h1>You can get data from user form here</h1>`);
}


module.exports = userDataSubmit;
```

This file:

1. Listens to data from the form
2. Collects data chunks using Buffer
3. Converts it to a readable object using querystring
4. Logs name and email in the console

Output Demo

node root.js

Open browser at:

http://localhost:3200/

19#########

n most programming languages, operations run **synchronously** — one task at a time. But in JavaScript and Node.js, we often use **asynchronous programming** for better speed and performance.

Synchronous Programming

1. Executes **one task at a time**.
2. The next line waits until the current one finishes.
3. **Slower** in terms of overall performance.

```
console.log("Apple1");

console.log("Apple2");

console.log("Apple3");
```

Each task waits for the previous one to complete.

Asynchronous Programming

1. **Doesn’t wait** for a task to finish before moving on.
2. Great for **I/O operations** (file, API, database).
3. **Faster** in real-world applications.

```
console.log("Apple1");


setTimeout(() => {
  console.log("Apple2");
}, 2000);


console.log("Apple3");
```

setTimeout() runs separately and doesn’t block the next line.

When Not to Use Asynchronous Code?

Sometimes, we want code to **wait** until one step is complete.

```
let a = 20;

let b = 0;


setTimeout(() => {
  b = 100;
}, 2000);


console.log(a + b); // Output: 20 (Not 120!)
```

Here, we **don't need** async because we want b before using it.

Real-Life Async Example

Reading a file without blocking the main thread:

```
onst fs = require("fs");


fs.readFile("text/peter.txt", "utf-8", (err, data) => {
  if (err) return false;
  console.log(data);
});


console.log("End Script");
```

Read FIle in Sync

```
const data = fs.readFileSync("text/peter.txt", "utf-8");
```

console.log(data);

20#########

Learn how to perform **Create, Read, Update, and Delete** operations using Node.js's built-in fs module. We'll also handle terminal inputs to do these operations dynamically.

Basic File Operations

const fs = require("fs");

Create New Files

fs.writeFileSync("files/apple.txt", "This is fruit");

fs.writeFileSync("files/banana.txt", "This is fruit");

writeFileSync() creates files. If file already exists, it will overwrite.

Delete a File

fs.unlinkSync("files/banana.txt");

unlinkSync() removes a file.

Read a File

const data = fs.readFileSync("files/apple.txt", "utf-8");

console.log(data);

readFileSync() reads file content. "utf-8" ensures readable string output.

Update a File

fs.appendFileSync("files/apple.txt", " and this is good for health");

appendFileSync() adds new content to existing file.

CRUD Operations with Terminal Input

Now let's make this dynamic — use terminal commands like Write, Read, update, or delete.

```
const fs = require("fs");
const operation = process.argv[2];
```

process.argv[] gives command-line input.

Create File from Terminal

node index.js Write fileName "your content"

```
if (operation === "Write") {
  const name = process.argv[3];
  const content = process.argv[4];
  fs.writeFileSync(`files/${name}.txt`, content);
  console.log("File Created");
}
```

Read File from Terminal

node index.js Read fileName

```
else if (operation === "Read") {
  const name = process.argv[3];
```

```
  const data = fs.readFileSync(`files/${name}.txt`, "utf-8");
  console.log(data);
}
```

Update File from Terminal

node index.js update fileName "new content"

```
else if (operation === "update") {
  const name = process.argv[3];
  const content = process.argv[4];
  fs.appendFileSync(`files/${name}.txt`, content);
}
```

Delete File from Terminal

node index.js delete fileName

```
else if (operation === "delete") {
  const name = process.argv[3];
  fs.unlinkSync(`files/${name}.txt`);
}
```

Fallback Case

```
else {
  console.log("Operation not found");
}
```

Example Test

node index.js Write hello "This is demo"

node index.js Read hello

node index.js update hello " updated!"

node index.js delete hello